Autonomous Navigation Using ArUco Fiducial Markers and Hazard-Aware Path Planning

***Abstract*— This work presents an autonomous navigation system for which an algorithm is designed to navigate a bot to a designated location and simultaneously detect and avoid hazards. Utilising an overhead camera, the developed system works with hazard identification and ArUco markers for Georeferencing and localisation. For identifying hazards, it utilises a CNN model YOLOv8, and path planning is processed by A* algorithm, constrained to identify path structures using masking and discretisation of the environment in a graph of traversable nodes representing Euclidean distances, which are communicated to the bot as direction commands. The bot is autonomous and is controlled by a calibrated PID controller. The bot prioritises the visits to hazards based on the shortest distance before navigating to the defined goal. The algorithm has a time complexity of *O(nlogn)* and requires *O(n)* space, where h and n represent number of hazards and the number of nodes, respectively.**

Keywords—A*algorithm, ArUco, Georeferencing, YOLO, Autonomous Navigation, Localisation, Hazard detection, CNN, PID.

1. INTRODUCTION

In the present world, robotics can be helpful in automating many activities, from industrial work to specialised tasks (ONO. K. et al.,2023). It has impacted people's lives in different fields such as medicine, space exploration, agriculture, and the military. Hence, to reduce workload and risk factors, many tasks are performed by machines. There is also a need for machines that can reduce physical labour and risk, but can independently identify and avoid Hazards. Notably the current growth of the computing industry, Iot products are becoming more affordable, making such systems easier and more cost-effective to develop. As people rely on eyesight to perceive the environment, incorporating vision into robotic systems, enabling real-time decision-making based on precepted input and other sensors, is able to enhance their ability to perform complex tasks. Initially used to control robot arms in real time to achieve desired configurations, cameras are now widely employed for real-time surveillance, object detection, classification, warning systems, and predictive tasks.

This is a scaled-down implementation of the real-world scenario using YOLO(You Only Look Once) and ArUco(Fiducial markers), which is discussed in section 2. It represents the foundational principles and core functions that can scale to real-world scenarios. It demonstrates how transferable components can interact with each other to produce an autonomous system capable of decision-making and adaptive navigation.

In this work, the problem statement is regarded as a multi-objective optimisation problem (MOOP) that requires an optimised shortest and hazard-free path. By utilising YOLO and Fiducial markers, a novel approach is developed for hazard detection and path planning. Fiducial marker and an autonomous navigation system are discussed under related work in Section II. Section III presents the system architecture, delineating the roles and interactions of the two core modules: the Physical bot module and computational module. Section IV describes the Methodology of Hazard detection, route planning, and Navigation algorithms. Section V shows the Implementation details explaining hardware and software, frame processing pipeline, and communication. In Sections VI and VII, discusses about the results and future development are discussed.

2. RELATED WORKS

The development of vision-based and autonomous navigation systems has leveraged a variety of technologies for environmental perception, localisation, and intelligent path planning. Recent advancements in the Iot sector and the wide availability of sensors and microprocessors have given more chances to such efforts. Such as in the work of(R. S. Rao et al.,2005) in which they present a framework for the development of a planar mobile robot controller based on image plane feedback. Their method relies on a waypoint-based trajectory generator, with all the waypoints specified in the image, as seen by an overhead observer . There are multiple papers about autonomous navigation, whether they are simulation-based, such as OpenDrive, as Simulators play a crucial role in autonomous driving, offering considerable time, cost, and labour savings. The categorisation of simulation based on feature functionalities delineates five primary classes: traffic flow, sensory data, driving policy, vehicle dynamics, and comprehensive simulators (Y.Li et al.,2024). SLAM, which stands for Simultaneous Localisation and Mapping, is a technique used in robotics and autonomous systems for mapping an unknown environment while simultaneously localising the robot's position within the map. One intuitive way of formulating SLAM is to use a graph whose nodes correspond to the poses of the robot at different points in time and whose edges represent constraints between the poses. The latter are obtained from observations of the environment or movement actions carried out by the robot (G. Grisetti et al.,2010).

Another way of perceiving the environment is by utilising augmented reality or a marker-based system. ArUco fiducial markers are notable examples(Sampathkrishna, A et al.,2022). For instance, in warehouse automation, robots use ArUco markers for their precise location and to achieve sub-centimetre accuracy(L. H. Pei,et al.,2022). ARtoolkit, Data Matrix, and ARSTudio are alternatives for ArUco. Their downside is, these systems suffer from occlusion sensitivity – if a marker is undetectable due to poor lighting or covered by a physical object. This can be mitigated by using redundant markers, but this increases the complexity of the system.

In comparison, learning-based perception methods, such as Convolution Neural Network, performs and offer better generalisation across unstructured environments. YOLOv4 is

an example of such an algorithm used for real-time object detection, making it suitable for detecting complex structures and dynamic environments as done in the work of (Zhao, H. et al.,2022), where Larger Scale 'You Only Look Once' Version 4 (LS-YOLOv4) algorithm for detecting the insulators and drop fuses. The YOLO extracts features of the given feed by a Convolutional Neural Network (CNN) and then performs feature fusion (Jiang, P. et al.,2022). Hybrid approaches are there to combine the strengths of both methods. For instance, a proposed system where a CNN refines pose estimation, when markers are partially occluded, improving robustness.

As for path planning algorithms, there are various methods, whether geometric model search methods, which include Dijkstra, A*, LPA*, D*, LSM, Random sampling methods, Bio-inspired methods, etc., as covered in "A Comprehensive Review on Autonomous Navigation" (S Nahavandi et al.,2022) and " Combined improved A* and greedy algorithm for path planning of multi-objective mobile robot"(Xiang. D et al.,2022)

3. System Architecture

The system is divided into two modules, one is the computational module, which can compute all the navigational and hazard detection of the system. This computational module is also divided into 3 subparts. The second module is a four-wheeled bot that takes direction of travel from the computational module. Each module operates in a coordinated pipeline flow.

3.1. Computational module

The computational module is the foundational component of the system, which can do the heavy lifting of vision processing, detecting the ArUco modules, locating the hazards, and creating a safe path to the defined goal.

3.1.1. Vision Processing Sub-Module

This section of the module is responsible for the initial video feed acquisition and processing all the precepted features of the maps, which are used for locating the hazards. The image acquisition is done by an overhead camera, which can capture at least 1080p video at 30 frames per second using a fixed mount such as a projector mount or an overhead mount. The camera is required to be at least 7ft above ground to simulate the drone footage. This consistent top view ensures spatial consistency, facilitating accurate localisation and path generation. ArUco markers for the coordinates of the map and for locating the initial position of the bot and the goal position. For detecting the markers, Opencv's built-in ArUco library is used to detect the defined fiducial markers. Every marker provides a unique ID and its position. Markers serve two purposes:

- Define reference points for georeferencing the environment.
- Represent hazard or goal locations.

It also serves the purpose of initial colour segmentation of the path as it has very distinct colour separation then the rest of the area. The regions are segmented using HSV(hue-Saturation-Value) colour space thresholding to create a binary mask.

3.1.2. Hazard Detection Sub-module

This module handles the detection of hazards using CNN, a custom model trained on a custom database to achieve accurate results. Single-layer CNN is used to gain speed and efficiency. Its core architecture enables it to simultaneously localise objects by predicting bounding boxes and classifying them into different classes of hazards. It also identifies path boundaries, providing corrective guidance if the bot deviates. Dynamic hazards are tagged using ArUco markers, with real-time coordinate updates to a hazard list. Upon detection, their world coordinates are immediately computed and added to a hazard list.

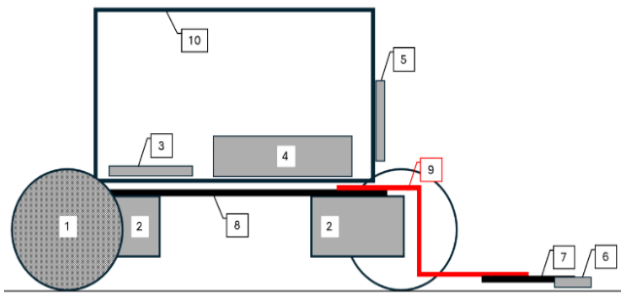
3.1.3. Path Planning Sub-module

This module plans collision-free, shortest routes to the goal while avoiding hazards. It achieves this by region mapping of the binary mask and transforming the path into a grid-based graph where each pixel or group of pixels corresponds to a traversable node(L Gehlod et al.,2013 ; L Gehlod et al.,2013). The regions that are not masked are marked as non-navigable zones and are excluded from the graph. The A* algorithm computes the shortest valid route to the goal and can dynamically replan the route if a new hazard has been detected on the map.

3.2. Hardware module

The hardware module that fully incorporates Iot development over the years. This wheeled autonomous bot, which is controlled by a System-on-a-chip microcontroller and a rover-based structure equipped with an Infrared Reflective tracking module, has its power source in the form of a battery pack and eliminating the need external source for power. It is built on a lightweight and sturdy sun board that can handle the weight of the battery and components easily. The four-wheel includes two driven wheels mounted using metal L brackets, which are bolted to the sun board in specifically drilled holes at each corner to maintain the alignment and drift of the track. Misalignments can be corrected by loosening the bolts. The motors are driven by a motor driver, which has PWM control so it can have variable rpm and can supply up to 12v to the motor.

Figure 1 Side View of the System



The front uses two metal double L mounts, which give it sturdiness while elevating it about 10mm above ground, as shown in Figure 1. It's also bolted to the sun board, and at the bottom part, a 5-array TCRT IR sensor is mounted low to the ground which detects the pure black regions on the path which are detectable by the track sensor so it can stay within the boundaries of path and can navigate around the curves and junctions. The tracker module has 5 individual active IR

sensors, which are placed 15mm with each other and work on the principle of infrared radiation.

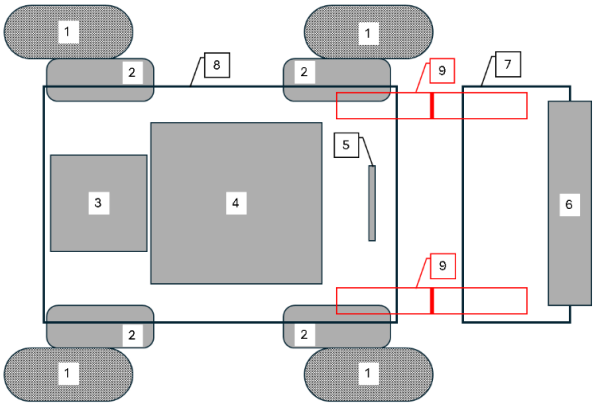


Figure 2 Top View of the System

Table 1 Component list

S. No.	Components
1	Wheels
2	Motor module
3	Motor driver
4	Battery pack
5	Microcontroller
6	Tracking sensor
7	Sensor base plate
8	Base plate
9	Double-L bracket
10	Case

Each sensor has an emitter part that emits infrared radiation and measures the reflected signals to determine the object's presence or distance. It provides both analogue and digital outputs, so it can use the distance given by each sensor, which is then used as the feedback for the PID controller, which adjusts the motor speed and direction of rotation to move the bot around. The combined sensor readings guide the bots' movement direction. All the PID calculations and sensor feedback are also managed by the microcontroller on the bot, which sends motor commands to the motor driver. Bot is equipped with Wi-Fi and Bluetooth; it0020manages communication with the computational module and receives from it. Figure 1 and Figure 2 illustrate the components and their placement.

On top of an ArUco marker, which has a unique ID, is placed as shown in Figure 3, so it can be recognised by the computational module. The bot is designed to be agile, precise, and can perform quick movements while in motion, but sturdy enough to handle the jerks of the movements and doesn't lose the alignment of its sensors.

4. Methodology

In this section, the methodology is discussed for detecting hazards and avoiding them using YOLOv8 and ArUco, as well as how ArUco is used for Homography of coordinates. How the path planning is performed by the A* algorithm, colour masking and filtering, and path following using PID and directions by the computational module.

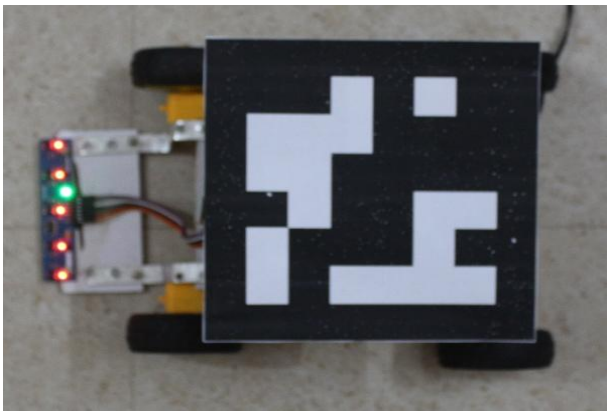


Figure 3 Top view Actual of the bot.

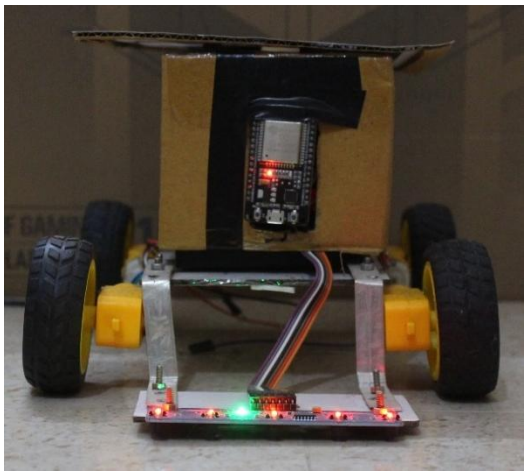


Figure 4 Front view of the bot.

4.1. ArUco and Georeferencing System

For recognition of the area, four ArUco markers are placed in each corner of the map with all having a unique ID ranging from 0,4,2,3, which are assigned to the real coordinate of that area. The bot and the goal are also determined by ArUco ID 5 and ID 7, respectively. The Four ArUco marker helps to define a coordinate reference frame, which forms a coordinate space. These values, which are abstracted from real-world units, establish a scalable and flexible coordinate system that can be used across environments. ArUco markers are a binary square with a fixed black border, which is mandatory for detection, and an internal binary code (black = 0, white =1), which is stored in the dictionary that can be obtained from the Opencv library. Moreover, the ARUCO.Dict_6x6_250 dictionary is utilised, where the 6x6 represents a unique binary matrix, so 6 bits in width and 6 bits of height (36 bits), and 250 is the number of unique patterns it can have, and depending on the unique ID, each bit is either black or white, forming a binary pattern , for more info please refer (Li, B. et al.,2020) The computational module captures the image, then finds all square contours in the image using edge detection and marks the candidates. Then, each marker is perspective-transformed to a flat frontal view, after which the internal 6x6 grid is extracted and binarised to read the bits and generate the ID. To ensure the camera detects all the markers, it gets calibrated, which is done using Opencv's standard calibration routine. Similarly camera's intrinsic matrix and distortion

coefficients are determined by analysing multiple views of a planar checkerboard pattern. It estimates the parameters necessary to map 3d world points to 2D image coordinates, correcting for lens-induced distortions.

The camera's intrinsic matrix K is defined as:
Equation 1

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Where:

- **fx, fy** are the focal lengths along the x and y axes (in pixels),
- **cx, cy** represent the optical centre (principal point).

After obtaining the coordinates from the ArUco marker, the pixel values from the overhead camera are converted to real-world coordinates using the homography transformation method. It creates a matrix ***H,*** which is used for the perspective transformation between the image plane (u,v) and world coordinates (x, y) shown in equation 2.

Mathematical Formulation

Given:

- **Image Points**: Detected marker centres in pixels.
- **World Points**: Known GPS coordinates of reference markers.

Solve for the homography matrix ***H*** (3×3) such that in:
Equation 2

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = H \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

Where:

- $h_{11}, h_{12}, h_{21}, h_{22}$ Are rotation, scaling and shearing factors.
- h_{13}, h_{23} Represent translation and h_{33} It is a normalising factor.
- h_{31}, h_{32} Are perspective distortion coefficients.

- Computed using **Direct Linear Transform (DLT)** with ≥4-point pairs.

For further details, please refer to (Dubrofsky, E.,et al.,2009)

After calculating the ***H,*** it performs the coordinate transformation, which converts any detected markers' pixel location to the real-world coordinate system., please refer (Agarwal,A.et al., 2007).

Mathematical Formulation

For a point (*x*,*y*) in the image, the converted value of the coordinate is obtained using Equation 3, where x', y' are converted points
Equation 3

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}, y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

This returns us the coordinates of the bot and area in the Universal Transverse Mercator (UTM), which is a standard coordinate system, X = Easting, Y = Northing. After which these are converted into WGS84 format, which is a standard for GPS, which gives out the latitude and longitude, which is stored in a Geojson file after every movement. This file is accessed by the GIS applications for displaying the live location of the area and the bot. Although this system is designed for demonstration and not scalable to real-world terrains (due to practical limitations of using visual markers), the conceptual approach mirrors the actual geolocation systems where landmarks, GPS, or SLAM techniques can serve the same purpose (Bláha, M. et al.,2012).

4.2. Hazard Detection

One of the core functions of this system is the autonomous identification of the hazards in the map. These hazards are detected, localised, and prioritised dynamically before the robot proceeds to the goal. To ensure robustness, the approach utilises CNN-based hazard classification. Specifically, a deep learning-based approach, **You Only Look Once version 8 - Small variant** model, is used for object detection(Duhan, S. et al., 2024). The YOLOv8 uses a custom CSPDarknet53 backbone, which helps in the information flow between layers, and a novel C2f (Cross Stage partial with two fusion paths) instead of the traditional Feature Pyramid Network (FPN). A pre-trained model of YOLOv8s is utilised, which demonstrated state-of-the-art performance on popular benchmark datasets, such as COCO and VOC. It is then fine-tuned for the database by the transfer learning process.

The model is trained in a batch size of 16 for 40 Epochs, optimised by SGD with momentum, and as for loss functions, CLou for box regression and BCE for class prediction is used. Each frame captured is preprocessed and passed through the YOLOv8s model. When any object is detected as a hazard class with confidence > 0.65, its bounding box centre is calculated and transformed into **world coordinates** using the homography matrix ***H***.

4.3. Path Planning and bot control

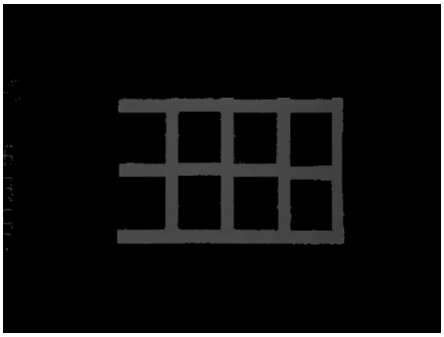


Figure 5 Colour Segmentation of the path (map 1)

After locating all the hazards and the position marker of the bot, goal the feed goes the color thresholding and creates a binary mask where pixels in BGR range upper bound [40, 30, 30] and lower bound [110, 90, 90] which was optimal for identifying navigable path as shown in figure 5.Navigable paths are transformed into nodes, each node represents a pixel coordinate or a grouped region, and then it is connected to its

4 or 8 neighbouring nodes, depending on adjacency (Ferguson, D. et al.,2007) . The bot first visits all the hazards and marks the visited hazards as well as the nodes **n** of the path which leads to the hazard in the order of priority, and returns to the start point after which it calculates the shortest path to the Goal.

Now the A* algorithm estimates the total cost f(n) for each node **n** nearest to the hazard or required to traverse to reach the hazard. For which the actual cost **g(n)** to and the heuristic cost **h(n)** from node **n** are estimated using Euclidean distance, and the total cost is determined for traversing. If a new hazard is introduced while traversing, the bot stops, recalculates the path, the continues again. This prunes the need to traverse back to the starting point for recalculation.

Mathematical Formulation:

- **Actual Cost:**

  g(n)=Cumulative distance from the start node to the goal

- **Heuristic Cost:**

Equation 4

$$h(n) = \sqrt{(x_{goal} - x_n)^2 + (y_{goal} - y_n)^2}$$

*This is the Euclidean distance to the goal, promoting shortest path discovery. Where (x_{goal}, y_{goal}) are goal position and (x_n, y_n) are current position

- **Total Cost:**

Equation 5

$$f(n)=g(n)+h(n)$$

For boosting performance in the environment, path caching is done where valid paths are stored temporarily and reused if still valid. Using cubic interpolation, it smooths out the turning curves, and Topological Simplification removes the intermediate nodes in a near-linear path (angle < 5°) as well as eliminates redundant chain nodes (degree = 2) for geometric clarity.

For giving direction commands to the bot, it computes the direction vector between the robot's current position and the next node **n,** which then gets normalised, and if $|\Delta x| > |\Delta y|$ It can turn left/right else it can move forward/backwards, which it can communicate over WI-FI using WebSocket.

Mathematical Formulation:

- Given path.

$$[p_0, p_1, p_2, \ldots.]$$

  Where: p_i represent the path

- Direction vector

Equation 6

$$\vec{d} = (p_1 - p_0) = (\Delta x, \Delta y)$$

Where: $(\Delta x, \Delta y)$Are the path difference points.

- normalise the direction:

Equation 7

$$\overrightarrow{d_{norm}} = \left(\frac{\Delta x}{\|\vec{d}\|}, \frac{\Delta y}{\|\vec{d}\|}\right)$$

- Where $\|\vec{d}\|$ Is the Euclidean norm:

Equation 8

$$\|\vec{d}\| = \sqrt{(\Delta x)^2 + (\Delta y)^2}$$

4.4. The Bot

The bot is controlled by the microcontroller, which manages all the sensors present on the bot, and equipped with Wi-Fi, it communicates with the computational module via WebSocket. The bot's movements are fully autonomous and are controlled by a PID controller. The PID controller is a system that receives input data from the tracker sensor, calculates the difference between the actual value and the desired value(Borase, R.P. et al.,2021). In this use case, it measures the relative position of the pure black region. It adjusts the output speed of motors to control the direction and the speed of the bot. PID operates within a closed-loop system; it continuously reads sensor feedback and adjusts outputs to minimise error. It achieves this through proportional control (**K_p**), which reacts to current error; integral control (**K_i**), which addresses accumulated past errors; and derivative control (**K_d**), which predicts future errors; it then sums these three errors to compute the output. Adjusting these values is called PID tuning to achieve the desired output and accuracy, which is explained in-depth in the work of (S. Bharat et al.,2019) for experimentally finding the optimum settings of the controller, in which it is possible to experimentally find the optimum values of the controller parameter, which is known as tuning.

4.5. Algorithm

The Pseudocode of the developed algorithm is illustrated below :

DNAV ALGORITHM

```
Input: Camera feed, ArUco markers, WebSocket connection
Output: Navigation path, robot state updates

Initialisation
robot_pos ← None, goal_pos ← None
hazard_positions ← ∅, visited_hazards ← ∅
ws ← ConnectWebSocket(WS_URL), cap ← OpenCamera()
aruco_dict ← LoadDictionary(DICT_6X6_250)
Main Loop
While camera is active do
    frame ← ReadFrame(cap)
    road_mask ← InRange(frame, ROAD_LOWER, ROAD_UPPER)
    (corners, ids) ← DetectMarkers(frame, aruco_dict)
DETECTION
```

```
if frame ≠ None then
    results ← YOLO.detect(frame)
    for each box in results do
        x1, y1, x2, y2 ← box.xyxy
        cls_id ← box.cls
        if cls_id ∈ HAZARD_IDS then
            hazard_center ← ((x1 + x2)//2, (y1 + y2)//2)
        hazard_positions ← hazard_positions ∪ {hazard_center}
        end if
    end for
end if
if ids ≠ None then
    H ← ComputeHomography(corners, ids, reference_markers)
    for i ← 1 to len(ids) do
        marker_id ← ids[i][0]
        pixel_pos ← Mean(corners[i][0])
        if H ≠ None then
            world_pos ← PerspectiveTransform(H, pixel_pos)
            (lat, lon) ← transformer(world_pos[0], world_pos[1])
            if marker_id = ROBOT_ID then
                robot_pos ← pixel_pos
                UpdateGeoJSON(marker_id, lat, lon)
            else if marker_id = GOAL_ID then
                goal_pos ← pixel_pos
            end if
        end if
    end for
end if
```

Navigation

```
if robot_pos ≠ None ∧ goal_pos ≠ None then
    unvisited ← hazard_positions \ visited_hazards
    if unvisited ≠ ∅ then
        h* ← argmin_{h ∈ unvisited}
                EuclideanDistance(robot_pos, h)
        path ← AStar(robot_pos, h*, hazard_positions, road_mask)
        if path ≠ None then
            cmd ← DetermineCommand(robot_pos, path)
            SendCommand(ws, cmd)
            if Distance(robot_pos, h*) < δ then
                visited_hazards ← visited_hazards ∪ {h*}
                SendCommand(ws, "STOP")
                Wait(1)
            end if
        end if
    else
        path ← AStar(robot_pos, goal_pos, hazard_positions,
road_mask)
        if path ≠ None then
            cmd ← DetermineCommand(robot_pos, path)
            SendCommand(ws, cmd)
            if Distance(robot_pos, goal_pos) < δ then
                SendCommand(ws, "STOP")
                break
            end if
        end if
    end if
end if
end while
```

Cleanup

```
ReleaseCamera(cap)
CloseWebSocket(ws)
```

Time Complexity Analysis of DNAV algorithm :

Theorem 1: (Time Complexity of A*)

Let T(n) denote the time complexity of the A* algorithm over an n area (i.e., n total grid nodes).
Then,

$$T(n) \in O(N \log n) = O(n \log n)$$

Derivation :
In each iteration, the A* algorithm selects the node with the minimum estimated total cost $f(n) = g(n) + h(n)$ from the priority queue (typically implemented using a binary min-heap). The core operations involved are:

- Insertion into the priority queue: $O(\log k)$
- Extraction (pop-min) from the queue: $O(\log k)$
- Priority update: $O(\log k)$

Where, k is the number of elements in the queue at any moment.

In this scenario, the algorithm may need to explore the entire area of size n, which includes n nodes. Each node is inserted and extracted at most once, with possibly several updates.
Hence, the total time can be modeled by the recurrence:
Equation 9

$$T(n) = T(n-1) + \log n$$

Solving the Recurrence
Expanding the recurrence:

$$\begin{aligned} T(n) &= T(n-1) + \log n \\ &= T(n-2) + \log(n-1) + \log N \\ &\vdots \\ &= T(1) + \sum_{k=2}^{n} \log k \end{aligned}$$

So,
Equation 10

$$T(n) = c + \log(n)!$$

Using Stirling's approximation:

$$\log(n)! \approx n \log n - n + O(\log n)$$

Thus,

$$T(n) = O(n \log n)$$

Conclusion
The time complexity of the A* algorithm over n nodes is:

$$T(n) = O(n \log n)$$

While other modules can contribute to its time complexity they are very small numbers in comparison to the nodes present in graph, hence they are lower order terms and ignored. So, the total time complexity comes out to be:
Equation 11

$$T_{DNAV} = O(n \log n)$$

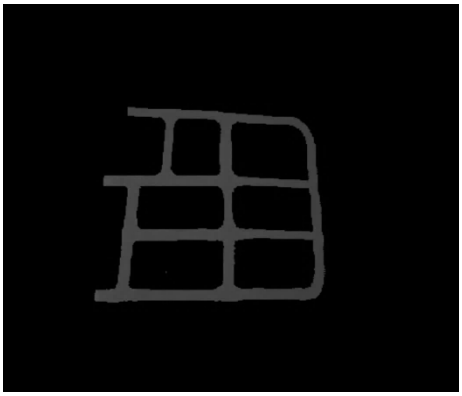


Figure 6 Path masking of the navigable area

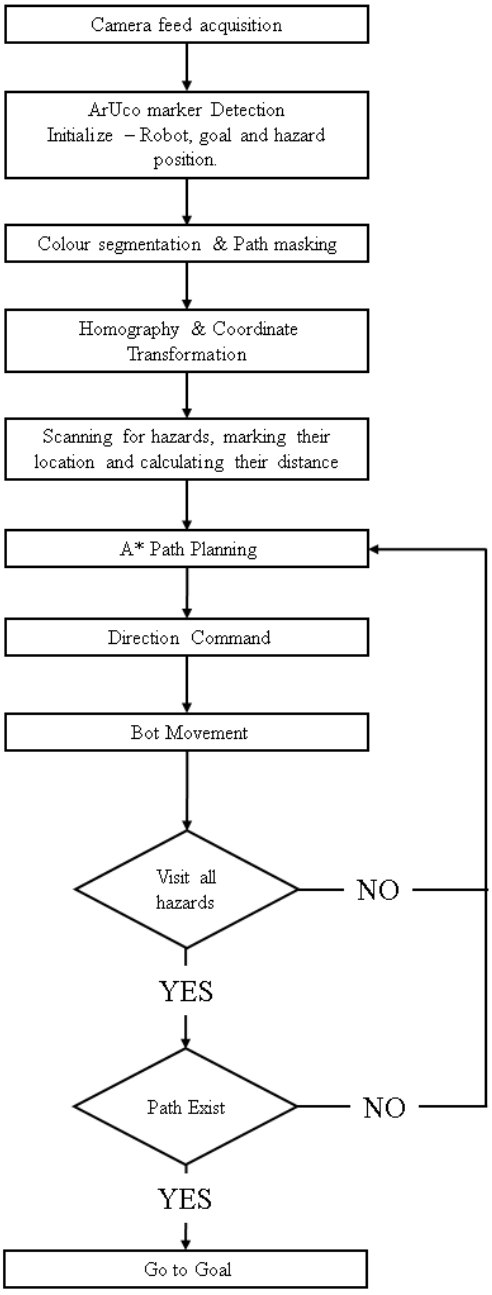


Figure 7 flowchart of the DNAV algorithm

As far as the DNAV algorithm explanation goes, several ordered steps occur as shown in Figure 6. In lines 5-8, the Camera, ArUco detection parameters, and dictionary are initialised, and make the connection with the microcontroller. Then, in lines 11-13, it creates the mask for the traversable path of the arena using colour segmentation shown in Figure 7 and the coordinates from the four ArUco markers, while also processing the initial position of the bot and the goal. This same mask is also utilised to isolate the navigable path from the invalid path in case it drifts away. It stores these mask pixel values, which are used in the navigation module.

Figure 8 GIS Software tracking the bot
Note: Red stars represents ArUco markers, the blue and red dot represents the bot initial position and goal position respectively.

After obtaining these values in line 16 it the detection module captures the Hazards present in the arena and in line 17-23 it calculates bounding boxes center, using the coordinates obtained earlier by the ArUco markers it calculates the position of the hazards in its world system using homography and coordinate transformation from line 27-33 and line 34-42 it updates the robot pos an goals pos and stores these values in a GeoJSON file (line 36) and keep updating it while the system is working, which then can read by GIS software for real time tracking as shown in figure 9

Once all hazards are located, in line 44, the bot is not in goal position, then from lines 45-49, it checks for hazards visited and measures the distance to the closest hazard and plans a path to visit the hazard. From lines 50-57, it sends movement commands and checks if it reached the hazard and marks its position and repeats this process until all hazards are visited. After which it goes to its starting position and plans the path to the goal, avoiding the hazards(from line 59-73).

After deciding the final path, it calculates the direction of travel using the orientation of the ArUco marker present on the bot and the coordinate values stored from the mask of the navigable path by calculating the normalised direction vector.

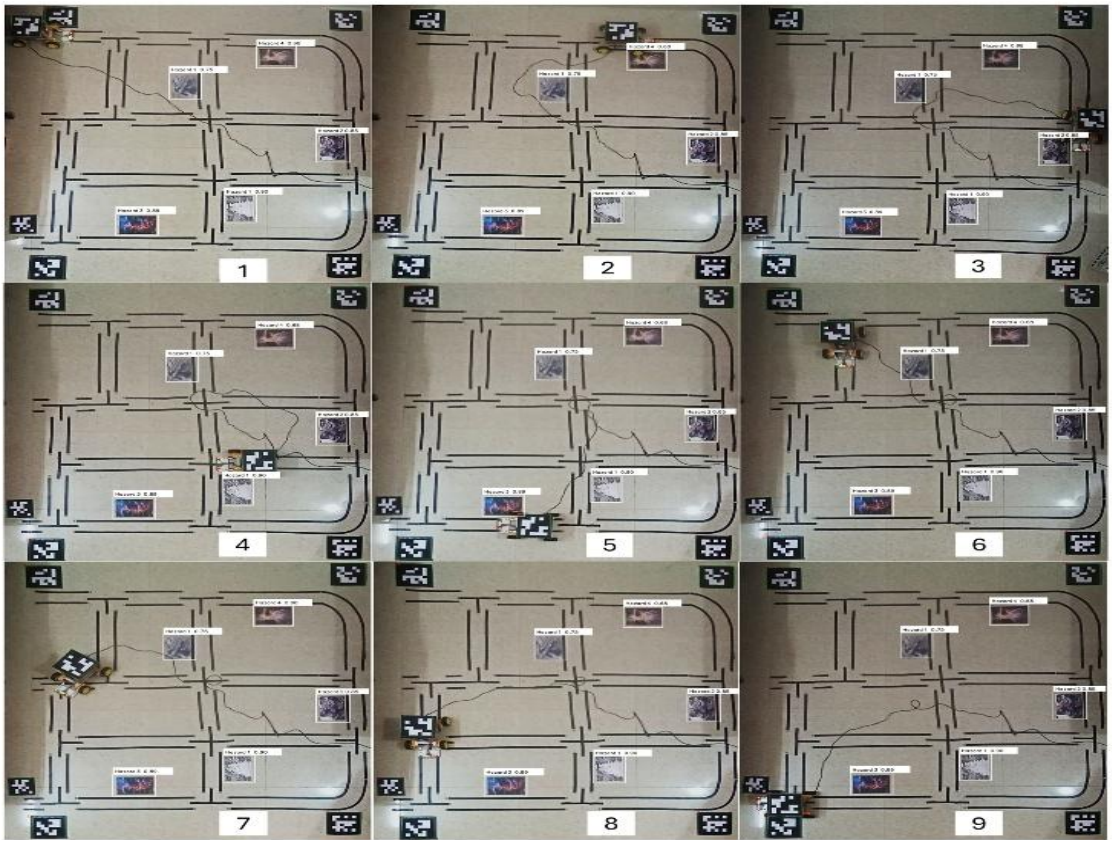


Figure 9 Bot navigating the arena

Based on the path, it then communicates these movement commands to the bot via Websocket until it reaches the goal position, as shown in Figure 9.

5. Result and Discussion

The system is evaluated across four primary dimensions: Detection Accuracy, Computational Efficiency, Navigational Optimality, and Localisation Accuracy. 5 hazards were placed on the map while system trials were taken as shown in Figure 10. Each metric was averaged over 35 independent trials.

Table 2 Database Images

The database has 7 hazard classes, and is split into a 70/20/10 training/validation/test ratio and annotated using a third-party annotation tool. The images are resized to 640x640 pixels to the default model image size. The data is collected from various open source image providers on the internet. It ranges from aerial strikes to house fires, which are more common in a suburban area or a high-density area compared to a natural environment as shown in Table 2.

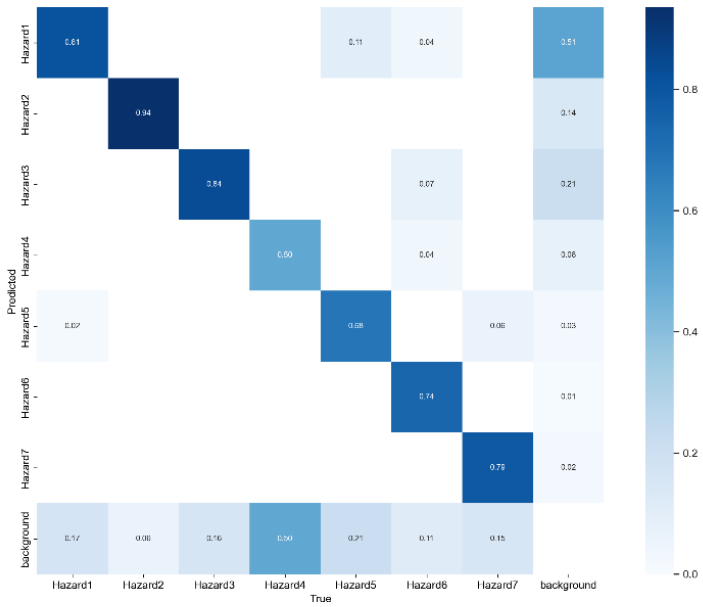


Figure 10 Confusion matrix.

Table 3 Comparison of methods of detection

Configuration	Precision	Recall	F1	False Positives/ min
ArUco Only	1.00	0.61	0.76	0
CNN Only	0.89	0.94	0.91	1.8
Hybrid	0.95	0.97	0.96	0.8

Table 3 presents the comparison for the hazard detection methods, whether in ArUco or CNN-only, or in Hybrid, where both are present. In addition, the Hybrid approach gives us better values, achieving 97% recall and 95% precision. And in Figure 10, the confusion matrix summarises the performance of the YOLOv8 model. The hybrid combination effectively handles marker occlusions, which is a limitation of ArUco, while on the other hand, it reduces false positives introduced by CNN-only inference.

Table 4 Model comparisons

Model	Params (M)	Accuracy	FPS
YOLOv8s	3.4	97.8%	30.4
EfficientNet-B0	5.3	94.1%	20
MobileNetV2	1.1	82.5%	28.9

As for the performance metrics of the model, different CNN models were trained on the same database and observed that the YOLOv8 performs better in terms of accuracy and provides much better frames per second, giving a robust performance in a real-world application as can be noted in Table 4 of model comparison.

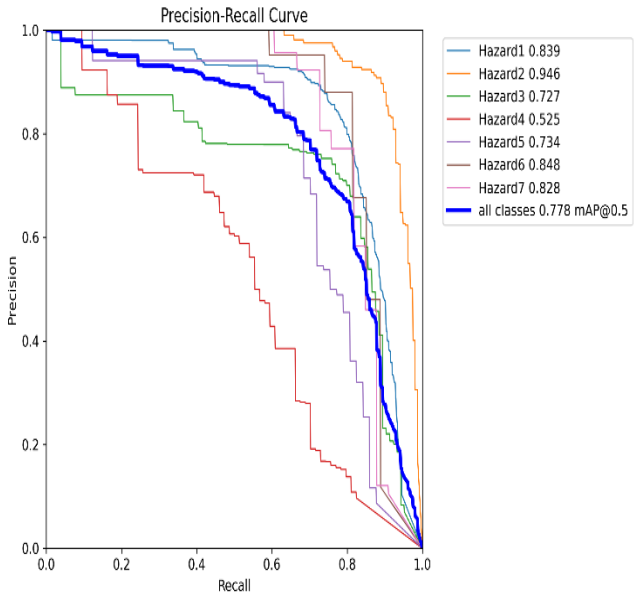


Figure 11 PR-curve of yolov8s

In Figure 11, the PR curve of the Yolov8 model, which achieves a mean average precision of 0.77, and for nearly all classes, is above 0.7, which is well above average in real-world applications.

Table 5 Cross module Timings

Operation	Worst-Case (ms)	Typical (ms)
Frame Capture	2.2	2.1
Color Segmentation	3.4	3.5
ArUco Detection	8.7	6.1
CNN Inference	22.4	15.6
Path Planning	6.3	3.8
Visualization	1.8	1.8
Total	44.8	32.9

The detection times per frame for the Yolov8 model were 32.9ms, in which the detection required for ArUco markers was 8.7ms, while achieving a sub-centimetre accuracy, which is enough to meet the requirements of a real-time application. Table 5 shows the worst-case and best-case execution times for each function of the system, and performs at 30 fps and 22.4 fps, respectively, which gives us an instant response system.

In the path planning section, the system presented solid sub-5-centimetre accuracy throughout the whole trials in localisation error, which was maintained consistently and is critical for ensuring safety during close-proximity navigation. From the perspective of the overhead camera, the maximum error on the x-axis was 3.2 cm and 4.1 cm on the y-axis.

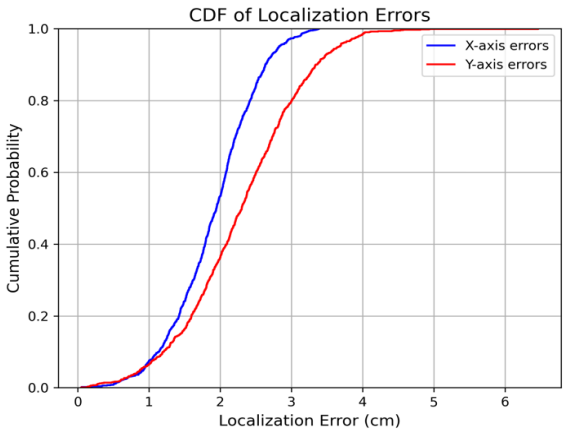


Figure 12: Probability graph of accuracy error

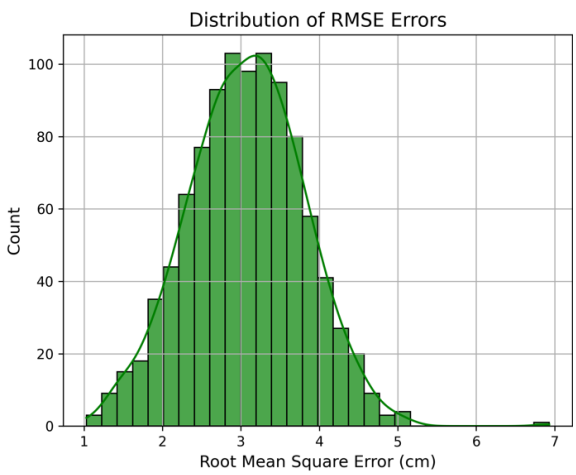


Figure 13:RMSE of Error intensity

As observable in the figure 12, that 95% of the time it keeps the error below the 5 cm through the whole run and from the figure 13 comply that most the errors are around the delta of 3.2±0.3 which shows the consistency of the system and the robustness of the hybrid ArUco marker and the homography system.

Table 6 Path Planning Metrics

Algorithm	Success Rate (%)	Path Length (m)	Replan Time (ms)
A*	100	5.21 ± 0.11	6.3 ± 0.9
Dijkstra	100	5.27 ± 0.15	14.7 ± 2.1
RRT	88	5.94 ± 0.23	22.1 ± 3.5

As for the actual planning, different algorithms were tested and concluded that A* meets the requirements. It has a 100% success rate of planning a path, with a short amount of time required to replan a new route. Dijkstra with heuristic also got similar results, but in terms of figures and time required to generate a new route is much slower. Likewise, RRT* and Informed RRT were tested, but the result was not as appealing as can be noted in Table 6

6. Conclusion and Future Work

There are three primary avenues for future research: (1) Currently, it relies on an overhead camera to capture the current environment, which can be replaced with an actual drone when implemented in real-life scenarios. (2) The current system is not suitable for the mobile robot to operate on tough terrains such as hills, sands, etc., and future work explores the hardware adaptations and control strategies. (3) Accordingly, it remains for further development to transition from ArUco to GPS-based localisation and implement SLAM and incorporate multi-sensors such as LIDAR systems for environments where marker-based approaches are not feasible. And how to improve the current algorithm via efficient operators, which can run on a mobile robot.

The developed system successfully demonstrates the effectiveness of the Algorithm employed for the detection of hazards and optimal path planning in a simulated environment using visual perception and georeferencing. The implementation of georeferencing using homography and ArUco ensures that every detection and movement is performed in a consistent manner, which is a key factor for achieving precise planning, localisation, and rapid response, creating a reliable system.

Experimental results prove the feasibility of the proposed approach is practicable in a real-world application, particularly in semi-structured environments where visual markers can be deployed. The system's modular architecture and performances across trials highlights its potential for larger, more complex scenarios, such as warehouse automation or smart city applications, where GPS or SLAM can replace fiducial markers.

7. References

1. Ono, K., Hayashi, T., Fujii, M., Shibasaki, N. and Sonehara, M., 2009. Development for industrial robotics applications. *IHI Engineering review*, *42*(2), pp.103-107.

2. R. S. Rao, V. Kumar and C. J. Taylor, "Planning and Control of Mobile Robots in Image Space from Overhead Cameras," Proceedings of the 2005 IEEE International Conference on Robotics and Automation, Barcelona, Spain, 2005, pp. 2185-2190,doi:10.1109/ROBOT.2005.1570437.

3. Y. Li et al., "Choose Your Simulator Wisely: A Review on Open-Source Simulators for Autonomous Driving," in IEEE Transactions on Intelligent Vehicles, vol. 9, no. 5, pp. 4861-4876, May 2024, doi: 10.1109/TIV.2024.3374044.

4. Xiang, D., Lin, H., Ouyang, J. and Huang, D., 2022. Combined improved A* and greedy algorithm for path planning of multi-objective mobile robot. *Scientific reports*, *12*(1), p.13273.

5. G. Grisetti, R. Kümmerle, C. Stachniss and W. Burgard, "A Tutorial on Graph-Based SLAM," in IEEE Intelligent Transportation Systems Magazine, vol. 2, no. 4, pp. 31-43, winter 2010, doi: 10.1109/MITS.2010.939925.

6. Sampathkrishna, A., 2022. ArUco Maker based localization and Node graph approach to mapping. *arXiv preprint arXiv:2208.09355*.

7. L. H. Pei, "Autonomous Mobile Robot Tracking Across Multiple Camera Vision," Bachelor's thesis, Faculty of Information and Communication Technology, Universiti Tunku Abdul Rahman, Kampar, Malaysia, Jan. 2022.Available: http://eprints.utar.edu.my/4654/1/fyp_CS_2022_LHP.pdf

8. Zhao, H., et al. (2022) 'Autonomous live working robot navigation with real-time detection and motion planning system on distribution line', *High Voltage*, 7(6), pp. 1204–1216. Available at: https://doi.org/10.1049/hve2.12221

9. Jiang, P., Ergu, D., Liu, F., Cai, Y. and Ma, B., 2022. A Review of Yolo algorithm developments. *Procedia computer science*, *199*, pp.1066-1073.

10. Saeid Nahavandi, Roohallah Alizadehsani, Darius Nahavandi, Shady Mohamed, Navid Mohajer, Mohammad Rokonuzzaman, and Ibrahim Hossain. 2025. A Comprehensive Review on Autonomous Navigation. ACM Comput. Surv. 57, 9, Article 234 (September 2025), 67 pages. https://doi.org/10.1145/3727642

11. Gehlod, Lalit, Mala Dutta, and Devesh Kumar Lal. "Graph Traversal technique for Grid Based robot Navigation." *International Journal of Advanced research in Computer and Communication Engineering* 2.7 (2013).

12. *Gehlod, L., Jain, V., Dutta, M. and Lal, D.K., 2013. A Grid Based Robot Navigation by Using Priority Algorithm. International Journal of Advanced Research in Computer and Communication Engineering, 2(8).*

13. *Li, B., Wu, J., Tan, X. and Wang, B., 2020, September. Aruco marker detection under occlusion using convolutional neural network. In 2020 5th international conference on automation, control and robotics engineering (CACRE) (pp. 706-711). IEEE.*

14. Dubrofsky, E., 2009. Homography estimation. *Diplomová práce. Vancouver: Univerzita Britské Kolumbie*, *5*

15. Bláha, M., Eisenbeiss, H., Grimm, D. and Limpach, P., 2012. Direct georeferencing of UAVs. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, *38*, pp.131-136.

16. Duhan, S. and Panwar, R., 2024. Yolo based vision-aided obstacles navigation and avoidance with path planning using sarsa algorithm for biped robot in an uncertain hospital environment. *Indian Journal of Science and Technology*, *17*(3), pp.236-246.

17. Borase, R.P., Maghade, D.K., Sondkar, S.Y. and Pawar, S.N., 2021. A review of PID control, tuning methods and applications. *International Journal of Dynamics and Control*, *9*, pp.818-827

18. Bharat, S., Ganguly, A., Chatterjee, R., Basak, B., Sheet, D.K. and Ganguly, A., 2019. A review on tuning methods for PID controller. *Asian Journal For Convergence In Technology (AJCT) ISSN-2350-1146*.

19. Ferguson, D. and Stentz, A., 2007, February. Field D*: An interpolation-based path planner and replanner. In *Robotics Research: Results of the 12th International Symposium ISRR* (pp. 239-253). Berlin, Heidelberg: Springer Berlin Heidelberg.

20. Agarwal, A., Jawahar, C.V. and Narayanan, P.J., 2005. A survey of planar homography estimation techniques. *Centre for Visual Information Technology, Tech. Rep. IIIT/TR/2005/12*.

Declarations

Conflicts of Interest and Informed Consent Declarations :

A verbal declaration stating - ***"All authors declare that they have no conflicts of interest"***